EXP.NO: 9	Banker's Algorithm
DATE: 13/03/2026	

AIM:

To implement the Banker's Algorithm for deadlock avoidance in C. The program aims to determine if a system is in a safe state after resource allocation and to find a safe sequence of process execution if one exists.

ALGORITHM:

1. **Preparation**: Calculate the **Need** for each process (`Max - Allocation`) and set all processes to "Not Finished."

2. **Selection**: Look for a process that is "Not Finished" and whose **Need** is less than or equal to the currently **Available** resources.

3. **Release**: If you find one, "pretend" it finishes. Add its **current Allocation** back into the **Available** pool and mark it "Finished."

4. **Conclusion**:

   If **all** processes finish, the system is **Safe**.

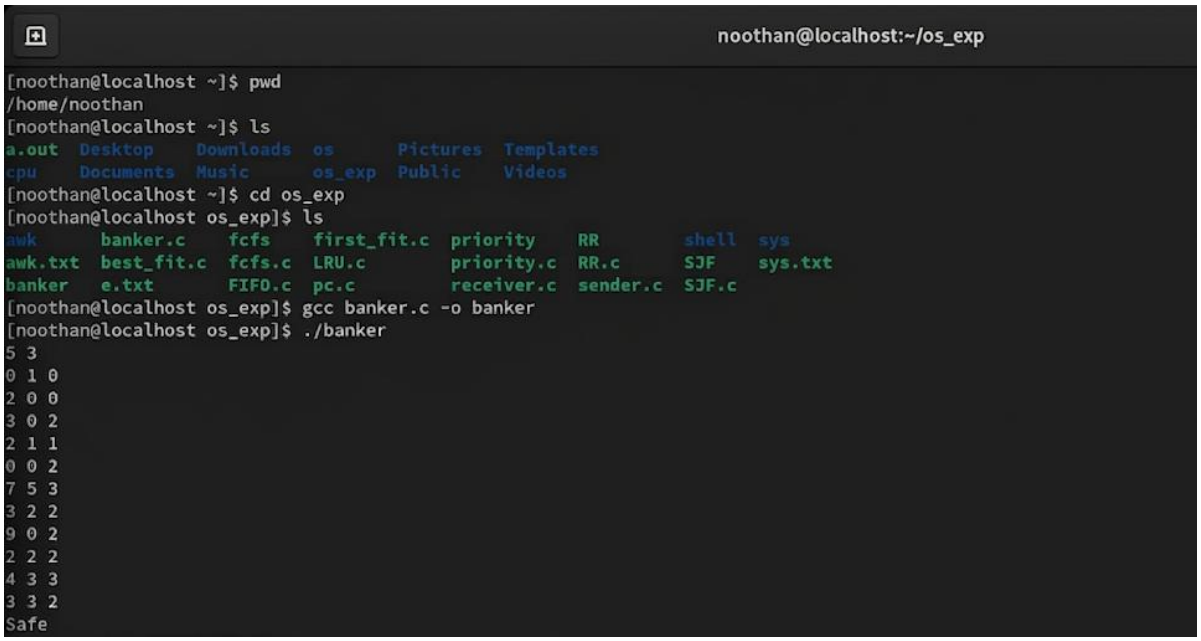
   If you get stuck and some are "Not Finished," the system is **Unsafe**.

CODE:

```
#include <stdio.h>

int main() {
   int n, r;
   scanf("%d %d", &n, &r);

   int alloc[n][r], max[n][r], need[n][r], avail[r], finish[n], seq[n];

   for (int i = 0; i < n; i++)
      for (int j = 0; j < r; j++)
         scanf("%d", &alloc[i][j]);

   for (int i = 0; i < n; i++)
      for (int j = 0; j < r; j++) {
```

```
            scanf("%d", &max[i][j]);
            need[i][j] = max[i][j] - alloc[i][j];
        }

    for (int j = 0; j < r; j++) scanf("%d", &avail[j]);

    for (int i = 0; i < n; i++) finish[i] = 0;

    int count = 0;
    while (count < n) {
        int found = 0;
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                int ok = 1;
                for (int j = 0; j < r; j++)
                    if (need[i][j] > avail[j]) ok = 0;

                if (ok) {
                    for (int j = 0; j < r; j++)
                        avail[j] += alloc[i][j];
                    seq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (!found) break;
    }

    if (count == n) {
        printf("Safe\n");
        for (int i = 0; i < n; i++) printf("P%d ", seq[i]);
    } else {
        printf("Not Safe");
    }

    return 0;
}
```

OUTPUT:



RESULT:

The program successfully takes inputs for n processes and r resources. It computes the Need matrix and simulates the Banker's safety check.